
UNIVERSITÉ DU SINE SALOUM EL HADJI IBRAHIMA NIASS**UFR des Sciences Fondamentales et de l'Ingénierie****Département de Mathématiques-Informatique**

Travaux Dirigés

Algorithmique et Programmation II

Thème : Les Pointeurs

Enseignant : Dr B. DIOP

Rappel : Dans ce TD, tous les exercices doivent être résolus en utilisant les **pointeurs** et l'**arithmétique des pointeurs**. L'accès aux éléments se fait via `*(ptr + i)` au lieu de `tab[i]`. Les fonctions reçoivent des pointeurs en paramètres.

Partie I : Pointeurs – Parcours et recherche**Exercice 1. Recherche des deux plus grands éléments**

Écrire une fonction qui reçoit un pointeur vers un bloc de 10 entiers et détermine, **en un seul parcours**, le plus grand et le deuxième plus grand élément.

Prototype :

```
void deuxPlusGrands(int *ptr, int taille, int *max1, int *max2);
```

Contrainte : Utiliser uniquement l'arithmétique des pointeurs pour parcourir les éléments.**Exercice 2. Vérification de l'ordre croissant**

Écrire une fonction qui reçoit un pointeur et une taille, et retourne 1 si tous les éléments pointés sont rangés en ordre croissant, 0 sinon.

Prototype :

```
int estTrieCroissant(int *ptr, int taille);
```

Indication : Comparer `*(ptr + i)` avec `*(ptr + i + 1)` pour chaque i .**Exercice 3. Comparaison de zones mémoire**

Écrire une fonction qui reçoit deux pointeurs vers des zones mémoire contenant des flottants et retourne le nombre d'éléments ayant **le même décalage et la même valeur**.

Prototype :

```
int compare(const float *p1, const float *p2, int size);
```

Test :

```
float notes1[] = {10.2, 12.5, 18.5, 9.8, 13.2, 12.1};  
float notes2[] = {15.7, 12.5, 15.3, 9.8, 10.8, 12.1};  
int resultat = compare(notes1, notes2, 6); // retourne 3
```

Exercice 4. Égalité de deux zones mémoire

Écrire une fonction `egal()` qui reçoit deux pointeurs et une taille. Cette fonction retourne :

- 1 si les deux zones mémoire contiennent des valeurs strictement identiques ;
- 0 sinon.

Prototype :

```
int egal(int *p1, int *p2, int taille);
```

Exercice 5. Vérification de sous-ensemble avec pointeurs

Écrire une fonction qui vérifie si tous les éléments pointés par `p2` sont présents dans la zone pointée par `p1`.

Prototype :

```
int estSousEnsemble(int *p1, int n1, int *p2, int n2);
```

Exemple :

```
int tab1[] = {3, 8, 7, 12, 6, 9, 1, 0, 5};
int tab2[] = {1, 3, 7, 6, 0};
int res = estSousEnsemble(tab1, 9, tab2, 5); // retourne 1
```

Exercice 6. Nombres successifs via pointeurs

Écrire une fonction qui reçoit un pointeur vers des entiers et vérifie si ces nombres sont consécutifs.

Prototype :

```
int sontSuccessifs(int *ptr, int taille);
```

Indication : Trier d'abord (ou parcourir après tri) et vérifier que `*(ptr+i+1) == *(ptr+i) + 1`.

Exercice 7. Recherche dichotomique avec pointeurs

Écrire une fonction de recherche dichotomique qui reçoit un pointeur vers une zone triée et retourne un pointeur vers l'élément trouvé, ou `NULL` si absent.

Prototype :

```
int* rechercheDicho(int *debut, int *fin, int valeur);
```

Indication : Utiliser `int *milieu = debut + (fin - debut) / 2;` pour calculer le milieu.

Partie II : Pointeurs – Statistiques et comptage

Exercice 8. Éléments uniques via pointeurs

Écrire une fonction qui parcourt une zone mémoire via un pointeur et affiche tous les éléments qui n'apparaissent qu'une seule fois.

Prototype :

```
void elementsUniques(int *ptr, int taille);
```

Exemple :

```
int tab[] = {5, 7, 7, 1};
elementsUniques(tab, 4); // Affiche : 5 1
```

Exercice 9. Fréquence des éléments avec pointeurs

Écrire une fonction qui compte et affiche la fréquence de chaque élément dans une zone mémoire pointée.

Prototype :

```
void frequences(int *ptr, int taille);
```

Indication : Utiliser un pointeur auxiliaire `int *p = ptr;` pour le parcours interne.

Exercice 10. Élément majoritaire via pointeurs

Écrire une fonction qui retourne un pointeur vers l'élément majoritaire (apparaissant plus de $n/2$ fois), ou NULL s'il n'existe pas.

Prototype :

```
int* elementMajoritaire(int *ptr, int taille);
```

Indication : Implémenter l'algorithme de Boyer-Moore avec des pointeurs.

Exercice 11. Différence pairs/impairs avec pointeurs

Écrire une fonction qui parcourt une zone mémoire et retourne la différence entre la somme des nombres pairs et la somme des nombres impairs.

Prototype :

```
int differencePairImpair(int *ptr, int taille);
```

Exemple :

```
int tab[] = {3, 5, 2, 4, 6};  
int diff = differencePairImpair(tab, 5); // retourne 4
```

Exercice 12. Plus petit positif manquant

Écrire une fonction qui trouve le plus petit entier positif manquant dans une zone mémoire non triée.

Prototype :

```
int plusPetitManquant(int *ptr, int taille);
```

Exemple :

```
int tab[] = {3, 1, 2, -5, 5, -23, 10, 6, 8, -7};  
int res = plusPetitManquant(tab, 10); // retourne 4
```

Partie III : Pointeurs – Modifications et réorganisation

Exercice 13. Suppression des doublons avec pointeurs

Écrire une fonction qui supprime les doublons d'une zone mémoire et retourne la nouvelle taille. Les éléments sont compactés en utilisant l'arithmétique des pointeurs.

Prototype :

```
int supprimerDoublons(int *ptr, int taille);
```

Indication : Utiliser deux pointeurs : un de lecture, un d'écriture.

Exercice 14. Insertion via pointeurs

Écrire une fonction qui insère une valeur à une position donnée en décalant les éléments via manipulation de pointeurs.

Prototype :

```
void inserer(int *ptr, int *taille, int position, int valeur);
```

Indication : Partir de la fin avec `int *p = ptr + *taille;` et remonter.

Exercice 15. Insertion dans une zone triée

Écrire une fonction qui insère une valeur dans une zone mémoire triée en maintenant l'ordre.

Prototype :

```
void insererTrie(int *ptr, int *taille, int valeur);
```

Indication : Trouver la position avec un pointeur, puis décaler.

Exercice 16. Suppression à une position via pointeurs

Écrire une fonction qui supprime l'élément à une position donnée et compacte la zone mémoire.

Prototype :

```
void supprimer(int *ptr, int *taille, int position);
```

Indication : Décaler avec `*(p) = *(p + 1);` dans une boucle.

Exercice 17. Déplacement des zéros avec pointeurs

Écrire une fonction qui déplace tous les zéros vers la fin de la zone mémoire en maintenant l'ordre des autres éléments.

Prototype :

```
void deplacerZeros(int *ptr, int taille);
```

Indication : Utiliser deux pointeurs : `int *lecteur = ptr;` et `int *ecrivain = ptr;`

Exercice 18. Conversion décimal-binaire avec pointeurs

Écrire une fonction qui convertit un entier décimal en binaire et stocke les bits dans une zone mémoire pointée.

Prototype :

```
int decimalVersBinaire(int nombre, int *bits);
```

Retour : Le nombre de bits écrits.

Partie IV : Pointeurs – Tri et fusion**Exercice 19. Tri à bulles avec pointeurs**

Écrire une fonction de tri à bulles utilisant uniquement des pointeurs pour accéder et échanger les éléments.

Prototype :

```
void triBulles(int *ptr, int taille);
```

Indication : Pour échanger :

```
int temp = *p;
*p = *(p + 1);
*(p + 1) = temp;
```

Exercice 20. Fusion de deux zones triées

Écrire une fonction qui fusionne deux zones mémoire triées en une troisième zone triée.

Prototype :

```
void fusionner(int *p1, int n1, int *p2, int n2, int *resultat);
```

Indication : Utiliser trois pointeurs qui avancent selon les comparaisons.

Exercice 21. Méli-mélo avec pointeurs

Écrire une fonction qui transfère les éléments d'une zone source vers une zone destination selon la règle :

- Éléments de rang impair : rangés en ordre inverse à la fin de la destination.
- Éléments de rang pair : rangés dans l'ordre au début de la destination.

Prototype :

```
void meliMelo(int *source, int *dest, int taille);
```

Partie V : Pointeurs et Matrices (Pointeurs de pointeurs)

Exercice 22. Détection des points creux

Écrire une fonction qui identifie les points creux d'une matrice allouée dynamiquement (pointeur de pointeurs).

Prototype :

```
void pointsCreux(int **matrice, int lignes, int colonnes);
```

Accès : $*(matrice + i) + j$ équivaut à $matrice[i][j]$.

Exercice 23. Parcours en accordéon

Écrire une fonction qui affiche les éléments d'une matrice (via `int **`) en zigzag.

Prototype :

```
void accordeon(int **matrice, int lignes, int colonnes);
```

Indication : Alternier le sens de parcours avec un pointeur qui va de $*(mat+i)$ à $*(mat+i) + col - 1$ ou inversement.

Exercice 24. Ligne avec le maximum de 1

Écrire une fonction qui retourne l'indice de la ligne contenant le plus de 1 dans une matrice binaire.

Prototype :

```
int ligneMaxUns(int **matrice, int lignes, int colonnes);
```

Exercice 25. Sommes des diagonales

Écrire une fonction qui calcule les sommes des deux diagonales d'une matrice carrée allouée dynamiquement.

Prototype :

```
void sommesDiagonales(int **matrice, int n, int *diagPrinc, int *diagSec);
```

Accès diagonal : $*(matrice + i)$ pour la principale, $*(matrice + i + n - 1 - i)$ pour la secondaire.

Exercice 26. Transformation triangulaire avec pointeurs

Écrire une fonction qui compte le nombre minimum d'échanges de lignes adjacentes pour obtenir une matrice triangulaire inférieure.

Prototype :

```
int versTriangulaire(int **matrice, int n);
```

Indication : Échanger les pointeurs de lignes : `int *temp = *(mat+i); *(mat+i) = *(mat+i+1);`
...

Partie VI : Allocation dynamique et gestion de données**Exercice 27. Gestion de notes avec allocation dynamique**

Créer un programme utilisant l'allocation dynamique pour gérer les notes d'étudiants :

- Allouer dynamiquement un tableau de chaînes : `char **noms = malloc(n * sizeof(char*))`;
- Allouer dynamiquement un tableau de notes : `float *notes = malloc(n * sizeof(float))`;

Écrire les fonctions suivantes :

```
float moyenne(float *notes, int n);  
int indiceMeilleur(float *notes, int n);  
void trierParMerite(char **noms, float *notes, int n);  
void libererMemoire(char **noms, float *notes, int n);
```

Partie VII : Problèmes avancés avec pointeurs**Exercice 28. Minimisation du produit scalaire**

Écrire une fonction qui réorganise deux vecteurs (via pointeurs) pour minimiser leur produit scalaire.

Prototype :

```
int produitScalaireMin(int *v1, int *v2, int n);
```

Indication : Trier `v1` croissant et `v2` décroissant, puis calculer $\sum_{i=0}^{n-1} (v1+i) \times (v2+i)$.

Exercice 29. Sous-séquences croissantes

Écrire une fonction qui compte le nombre de sous-séquences strictement croissantes d'une zone mémoire.

Prototype :

```
long long compterSousSequences(int *ptr, int n, int modulo);
```

Indication : Utiliser la programmation dynamique avec un tableau alloué dynamiquement.

Exercice 30. Groupement par facteurs premiers

Écrire une fonction utilisant des pointeurs et l'allocation dynamique pour déterminer le nombre d'ensembles distincts après fusion par facteurs premiers communs.

Prototype :

```
int nombreEnsembles(int A, int B, int P);
```

Indication : Utiliser une structure Union-Find avec allocation dynamique.

Exercice 31. Soudoyer les prisonniers

Écrire une fonction qui détermine l'ordre optimal de libération minimisant le coût total.

Prototype :

```
int coutMinimal(int P, int *prisonniers, int Q);
```

Indication : Utiliser la programmation dynamique avec une matrice allouée dynamiquement.

Exercice 32. Centre de masse d'un essaim

Écrire une fonction qui calcule la distance minimale du centre de masse à l'origine pour un essaim de lucioles.

Structures et prototype :

```
typedef struct {
    double x, y, z;
    double vx, vy, vz;
} Luciole;
```

```
void centreMinimal(Luciole *essaim, int n, double *dMin, double *tMin);
```

Exercice 33. Couverture par carrés

Écrire une fonction qui vérifie si une grille peut être entièrement couverte par des carrés 2×2 .

Prototype :

```
int couverturePossible(char **grille, int lignes, int colonnes);
```

Exercice 34. Harmonie parfaite

Écrire une fonction qui trouve une fréquence en harmonie avec toutes les autres.

Prototype :

```
int trouverHarmonie(int *frequences, int n, int L, int H);
```

Retour : La fréquence trouvée, ou -1 si impossible.

Exercice 35. Plan de voyage spatial

Écrire une fonction qui maximise la consommation de carburant pour un voyage spatial.

Prototype :

```
int voyageOptimal(int *positions, int n, int carburantMax);
```

Exercice 36. Chemins du Yin et du Yang

Écrire une fonction qui compte le nombre de grilles valides où chaque couleur forme un chemin.

Prototype :

```
long long compterGrillesValides(int lignes, int colonnes);
```

Exercice 37. Simulation de bactéries

Écrire une fonction qui simule l'évolution des bactéries et retourne le temps d'extinction.

Prototype :

```
int tempsExtinction(int **grille, int lignes, int colonnes);
```

Exercice 38. Intervalles intéressants

Écrire une fonction qui détermine si un intervalle contient un nombre pair de palindromes.

Prototype :

```
int estInteressant(long long L, long long R);
```

Bon courage !